Datawhale

动手学大模型应用开发 · 第 8 节

综合实践——资料与仓库导读

建立 PDF 与代码仓库学习助手基线



主讲：陈思州

本节学习路线

Datawhale

综合项目第一阶段 · 先交付可核查的资料导读

第八节建立可用基线，第九节再增加练习、反馈和学习历史。

01 项目背景

学习者、资料输入、目标与安全边界

02 组合能力

检索、上下文、MCP、Skill、Harness 与评估

03 导读基线

资料地图、证据包、学习路径和状态

04 真实效果

保存结果、事实核验、失败与改进项

先证明系统能基于固定资料完成一次有证据的导读，再增加学习闭环。

综合项目：学习助手目标统一，技术路线自定

Datawhale

作业边界 • 同一套验收要求允许不同技术路线

课程示例负责把方法讲完整，不要求学习者复制同一个界面、目录或框架。

个人 AI 学习助手　　PDF 与代码仓库导读、练习反馈、状态保存和跨仓库迁移

01 自选学习场景

使用明确授权的资料，写清学习者、目标和完成条件

02 完成可运行实现

Notebook、CLI、网页或框架均可，关键路径可复现

03 提交可复核证据

报告 PDF + 代码仓库；运行与失败证据放在仓库

04 换资料重新验收

更换资料或任务，重新建立坐标、条件和回归用例

实现形式可以自由选择，任务、证据、失败分析和迁移结果必须能够检查。

综合项目：PDF 与代码仓库学习助手

Datawhale

项目背景 • 帮助学习者从“资料很多”走到“下一步可执行”

系统面向明确授权的本地资料，不执行第三方仓库，也不把检索命中当成事实正确。

01 学习者目标

已有基础、想学内容、时间与完成条件

02 资料输入

固定版本 PDF、README、源码和示例数据

学习助手

围绕一个学习目标组织资料、证据、步骤与运行状态

03 系统处理

资料地图、检索、读取、证据包和上下文

04 可检查交付

带来源解释、学习路径、自检与进度产物

综合案例把前七节知识放进一个能运行、能核查的学习任务。

前七节在项目中各自承担一项责任

Datawhale

能力组合 · 每一章都在项目中留下一个可观察接口

系统效果来自部件协作，不能只展示最后一段模型回答。

01	模型与 Agent	理解目标，选择读取、检索或回答
02	检索与上下文	找到候选证据并组织本轮输入
03	MCP 与 Skill	提供只读资料工具和导读方法
04	Harness	保存状态、预算、产物与恢复边界
05	评估	核对引用、事实、交付物和失败原因

第八节把分散能力连接成端到端导读基线。

第八节建立可用的资料导读基线

Datawhale

第一阶段范围 · 先完成可复现导读，再扩展学习闭环

每一块都有可检查产物，避免“接了模型”成为唯一完成标准。

第八节基线　　从固定资料输入到带来源学习路径

01 只读接入

来源、版本、哈希、PDF 页与源码行

02 证据构建

资料地图、检索、全文读取和证据包

03 学习路径

步骤依赖、阅读动作、自检与完成条件

04 运行验收

预算、状态、产物、引用与失败记录

第八节交付的是可核查导读系统，不是一个只会生成摘要的聊天窗口。

项目画面：第一章资料的真实导读

Datawhale



实际网页画面 · 已保存的真实模型回答

这一轮导读怎样验收

输入可替换

本轮使用第一章 PDF 与自编仓库，系统接口可接新资料

来源可回查

回答引用 PDF 物理页与 agent.py、tools.py

解释要审阅

引用读回成功不等于每句话都被原文支持

真实回答、已读证据与审阅问题一起构成可检查的导读结果。

资料接入先建立身份，再提取和索引

Datawhale

接入管线 • 原始来源是证据，索引只是查找结构

PDF、代码和 Markdown 的提取方法不同，但都需要版本、位置和内容身份。

01	**登记来源**	文件、仓库、许可与可见范围
02	**固定版本**	文件哈希或仓库提交
03	**提取内容**	文本、代码、结构与元数据
04	**建立坐标**	页码、路径、行号与块范围
05	**生成索引**	关键词、向量或结构关系

先能说明资料是谁、来自哪里，再讨论怎样搜得更准。

本项目怎样处理文本页、扫描页和图形

Datawhale

课程实现 • 机器可读文本走自动索引，扫描和图形保留原页证据

示例 PDF 可直接提取文本；扫描页和图形保留为扩展路径，当前不做 OCR。

01	逐页提取	PdfReader 打开文件，对每个物理页调用 extract_text
02	判断能否索引	空文本页写入 empty_pages；有文本的页面继续切块
03	保存定位信息	十二行一块、重叠两行，保留页码、行范围与 SHA-256
04	处理扫描与图形	pdftoppm 渲染原页，再接 OCR、版面分析或人工复核
05	回查原页	记录处理方法、区域与质量标记，关键结论回到原图核验

当前项目真实完成文本提取和空页报告；扫描与图形路径明确保留，但不冒充已经识别。

PDF 与代码仓库需要不同定位坐标

Datawhale

A

PDF 文档

文件哈希、物理页码、提取文本行范围和文本哈希。扫描页没有文本时标记需要 OCR。

边界

B

代码仓库

远程来源、提交哈希、相对路径、源码行范围和文件哈希。分支名不能替代固定提交。

代码仓库需要结构图而不只是文件列表

Datawhale

仓库地图·文件位置之外还要表达运行关系

目录树只能说明文件在哪里，不能说明系统从哪里启动、怎样流动以及哪些行为受测试保护。

可运行的仓库地图 入口、依赖、配置、测试和版本共同组成阅读坐标

01	入口与命令	启动脚本、CLI、服务入口、安装和常用运行方式
02	模块与依赖	import、调用、数据流和关键接口之间的关系
03	配置与数据	环境变量、配置优先级、输入输出和生成产物
04	测试与变更	行为边界、失败用例、版本标签和相关提交

仓库地图先回答怎样运行，再回答每个文件讲什么。

PDF 只是一个例子：不同资料需要不同回查坐标

Datawhale

资料形态	接入、定位与验证
文档与表格	固定文件版本，保存页码、段落、行列或工作表坐标；回到原文件核对
代码仓库	固定提交，保存路径、符号和行范围；结合测试、调用链和运行结果验证
网页与接口	保存网址、获取时间、请求参数和响应字段；用快照或原始响应留证
图片与音视频	保存原文件、区域或时间点和识别方法；重要内容需要人工或第二种方法复核

证据单元同时保存内容、位置、版本与身份

Datawhale

证据数据模型 · 检索命中的不是匿名文本块

证据单元要支持查找、展示、引用、版本失效与重新索引。

Evidence Unit —一个可搜索、可读取、可回查的证据对象

01	内容	原文、代码或结构化片段
02	位置	页面、路径、行号与块范围
03	版本	文件快照、提交与提取版本
04	内容身份	稳定 ID、哈希与来源引用

内容变了，旧证据应失效或保留为历史版本，不能悄悄指向新文本。

资料地图与检索索引承担不同职责

Datawhale

A

资料地图

描述来源、结构、入口、依赖和版本。帮助理解资料空间，适合规划先读哪里。

边界

B

检索索引

根据当前问题返回候选证据单元。帮助缩小搜索范围，适合为一项具体结论补充原文。

项目画面：PDF 与源码都能回到原文

Datawhale



实际网页画面 • 第一章 PDF + 自编仓库

从画面里检查资料入口

输入身份

7 份文件、102 个片段，PDF 与源码同属一个资料版本

位置可回查

PDF 用物理页，源码用文件和行范围

检索有边界

相关度只选候选；结论要回到完整原文

网页实物展示的是当前资料的检索结果，换资料后这些坐标与结论必须重算。

复杂问题要拆成可取证的子问题

Datawhale

问题规划 • 每个子问题都要对应可定位证据

一次宽泛检索容易返回主题相关但无法共同支持完整结论的片段。

1. **界定目标**
   说明要解释的行为、版本和适用范围
2. **列出证据需求**
   拆出定义、入口、过程、结果和边界
3. **分别检索**
   为每项需求寻找并读取决定性原文
4. **合并冲突**
   按版本和来源解释一致、补充或矛盾关系
5. **检查缺口**
   标记尚未读取或无法确认的环节

问题分解的输出不是更多查询，而是一张证据需求清单。

证据包要把结论、原文和边界绑定

Datawhale

回答输入 • 让每个重要判断都能沿来源回查

引用存在只能证明有出处；还要检查证据是否覆盖并真正支持对应结论。

Evidence Packet 本轮回答实际可用的已读证据

01 结论

本轮需要解释或验证的命题

02 证据

原文、代码、运行结果或冲突

03 来源

版本、位置、ID 与内容哈希

04 边界

推断、缺口、歧义与待复核项

引用合法、内容一致、语义支持是三项不同检查。

证据包进入上下文时仍要重新编排

Datawhale

上下文装配 · 从证据集合变成可阅读的任务输入

检索命中并不决定模型的阅读顺序，排列和标签会影响证据能否被正确使用。

受预算约束的证据上下文　完整保留决定性条件，外部化低频和大体量材料

01　**任务说明**　当前问题、资料版本、范围和不能外推的边界

02　**决定性证据**　支持关键结论的原文、源码或运行结果

03　**关系与冲突**　按依赖、对照和版本排列，并标记不一致

04　**输出契约**　要求引用、事实与推断分层，并允许资料不足

上下文要让证据关系可见，而不只是让片段全部出现。

学习路径要表达依赖，而不是目录改写

Datawhale

路径设计 • 每一步都说明为什么现在学、学完能做什么

一条路径不是把目录换成自然语言，而是把依赖、证据、行动与验收连起来。

01	**明确起点**	目标、基础、时间与资料范围
02	**排列依赖**	概念、源码与运行前置关系
03	**绑定资料**	页面、文件、函数与证据 ID
04	**设计动作**	阅读、运行、修改或解释
05	**设置自检**	用问题或结果验证理解

好的路径让学习者产生可检查的新能力，而不只是读完更多文字。

学习步骤需要动作、产物与完成条件

Datawhale

学习路径 · 每一步都产生可检查结果

只有章节顺序的路径无法判断学习者是否理解，也无法支持中断恢复。

可验收学习步骤　　目标、资料、行动、产物和完成条件相互绑定

01 阅读动作

带着问题读取指定版本和位置

02 实践动作

运行、修改、比较或解释一个具体行为

03 保存产物

留下笔记、命令输出、截图、代码或结论

04 完成判定

用自检、测试或口头解释确认目标达成

路径的最小单位是一项可执行、可保存、可验收的学习活动。

导读任务需要状态、产物与验收协作

Datawhale

持续学习 · 路径会变化，已验证成果和边界不能丢失

状态用于推进，产物用于复用和验收，消息只承担本轮交流。

1. **保存目标**
   学习成果、资料版本与预算
2. **推进步骤**
   读取证据并执行当前动作
3. **保留产物**
   解释、代码结果、自检与来源
4. **接收反馈**
   理解状态、困难与新问题
5. **重新验收**
   更新未完成部分并核对结果

恢复的是有证据的学习状态，不是无限增长的聊天历史。

资料版本变化会使部分证据失效

Datawhale

资料新鲜度 · 变化传播到证据和学习路径

系统能够重新打开文件，不代表旧引用仍指向相同内容。

01	检测版本	比较提交、文档哈希、修改时间或发布版本
02	定位影响	查找引用了变化文件、页面或符号的证据包
03	标记旧结论	保留历史依据，同时停止把它当作当前事实
04	重新取证	读取变化位置及其依赖，重新判断支持关系
05	更新路径	只重开受影响步骤，保留仍有效的学习成果

版本变化要触发选择性重验，而不是全盘遗忘或继续沿用。

迁移到新资料时复用流程，不复用旧结论

Datawhale

A

可以复用

导入、版本固定、检索读取、证据包、状态保存、引用核验和任务验收流程。

边界

B

必须重做

资料地图、索引、坐标、关键分支、事实结论、支持关系和适用边界。

验收要沿交付物逐层回查证据

Datawhale

验收链·从可定位来源一直检查到新资料上的真实结果

系统运行完成、引用格式合法和学习任务完成是三个不同结论。

01	**核对来源**	版本、坐标、哈希与可访问性
02	**核对事实**	结论是否由对应证据支持
03	**核对路径**	步骤、依赖、动作与自检可执行
04	**核对状态**	进度、预算、产物与恢复一致
05	**核对迁移**	新资料重新索引并通过同类任务

交付物可回查、可执行、可恢复、可迁移，才构成完整导读系统。

压力测试要主动寻找证据链断点

Datawhale

反例验收 • 可靠系统也要证明自己何时拒绝回答

只用顺利样例会掩盖资料不足、版本冲突和错误复用。

证据链压力测试　　用可复现反例检查定位、取证、回答和迁移边界

01 无答案问题

资料不包含结论时，回答是否明确保留未知

02 冲突版本

不同版本陈述不同时，是否展示版本和冲突

03 抽取错序

双栏、表格或 OCR 错误是否触发原页核验

04 错误迁移

旧坐标或旧结论进入新资料时是否被拒绝

系统的可靠性也体现在它能识别证据不足并停止。

本节知识结构

Datawhale

1. **课程案例用 PDF 与代码仓库展开，核心方法可迁移到其他资料。**
2. 第八节完成只读接入、资料地图、证据包、上下文、状态与引用核验。
3. 真实记录已经生成三步导读，也暴露篇幅、前提和解释边界问题。
4. 第九节沿用资料底座，增加练习执行、反馈、学习历史和迁移回归。



参考文献与延伸阅读

Datawhale

本节关于资料导读、来源定位、版本固定与证据追溯主要参考以下论文和官方资料。

[1] Lewis, Patrick, et al.
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
NeurIPS 2020 · arXiv:2005.11401

[2] Anthropic
Effective Context Engineering for AI Agents
anthropic.com/engineering/effective-context-engineering-for-ai-agents

[3] pypdf
Extract Text from a PDF
pypdf.readthedocs.io/en/stable/user/extract-text.html

[4] Git
Revision Selection
git-scm.com/book/en/v2/Git-Tools-Revision-Selection

[5] W3C
PROV-O: The PROV Ontology
w3.org/TR/prov-o